Práctica: frontend React con lo visto hasta el paso 7

Práctica 1. Ver el detalle de un usuario

Al hacer clic en un nombre de la lista, muestren su email, edad, país y roles.

- **Endpoint:** `GET /user/{id}`.
- **Pista:** guarden en un estado el `id` elegido y usen un `useEffect` que dependa de él.
- **Para pensar:** en Network, comparen la respuesta de `GET /user` con la de `GET /user/{id}`. La lista trae de cada usuario solo `id` y `name`, y el detalle trae todos los datos de uno solo. ¿Por qué la API no manda todos los datos de todos los usuarios en la lista y se ahorra el segundo pedido? Piensen qué pasaría con 10.000 usuarios.

Práctica 2. "Cargando..." y errores

Mientras llega la lista, muestren `Cargando...`. Si la API responde con error, muestren el `detail` en lugar de la lista.

- **Pista:** dos estados más, `loading` y `error`, y miren `response.ok` como en el paso 4.
- **Para probar el error:** cambien la URL por `/usr` y debería aparecer `Not Found`. Pongan en Network la velocidad *Slow 4G* para ver el `Cargando...`.

Práctica 3. Buscar por nombre

Un input y un botón **Buscar** que muestren solo los usuarios que coinciden.

- **Endpoint:** `GET /user/search/{name}`. Busca por parte del nombre: prueben con `ma`.
- **Pista:** usen `encodeURIComponent(name)`. Busquen `#` con y sin usarlo, y comparen en Network la URL que sale.
- Agreguen un botón **Limpiar** que vuelva a la lista completa.

Práctica 4. Mostrar mis roles

Cambien el saludo del paso 6 por `Hola, Martina (ADMIN)`.

- **Pista:** la respuesta de `GET /user/{id}` ya trae `roles`. Amplíen la interfaz `User` y usen `.map(...).join(', ')`.

Práctica 5. Alta de usuario

Un formulario con nombre, edad, email, contraseña y país que cree un usuario.

- **Endpoint:** `POST /user`, con body `{ name, age, email, password, country_id }`.
- **Pistas:**
- El país es un `<select>`: no hay endpoint de países, así que escriban las opciones a mano (1 Argentina, 2 Brasil).
- `age` y `country_id` tienen que viajar como número: usen `Number(...)`.
- Usen `type="email"` y `minLength={4}` en la contraseña.
- Cuando el alta salga bien, actualicen la lista sin recargar la página.
- **Con `sfer`:** ¿qué responde la API? Muestren ese mensaje.

Práctica 6. Solo mayores de edad

Un botón que alterne entre **Todos** y **Solo mayores**.

- **Endpoint:** `GET /user_mayores`.
- Todos los usuarios de prueba son mayores. Con la práctica 5, creen uno de 16 años y comparen las dos listas.

Práctica 7. Borrar un usuario

Un botón **Borrar** en cada fila.

- **Endpoint:** `DELETE /user/{id}`.
- Pidan confirmación con `window.confirm(...)` y saquen el usuario de la lista sin volver a pedirla. **Pista:** usen `filter`.
- **Con `sfer`:** la API responde `403 Missing permission: delete_users`. Muestren el mensaje. ¿Les parece bien que el botón aparezca igual? Lo resolvemos en el paso 12.
- Borren solo los usuarios que crearon ustedes.

Práctica 8. Sesión vencida

Si cualquier pedido responde **401**, borren el token y vuelvan al login con el mensaje `Tu sesión venció`.

- **Para probarlo sin esperar 30 minutos:** en *Application → Local Storage*, cambien una letra del token y recarguen.
- **Pista:** la función que cierra la sesión está en `App`, y el `fetch` en `UserList`. ¿Cómo le avisa uno al otro? Pasen una función como prop: `onUnauthorized`.

Práctica 9. Para pensar antes del paso 8

Con los ejercicios hechos, cuenten en su código cuántas veces escribieron:

- `import.meta.env.VITE_API_URL`
- `Authorization: \`Bearer ${token}\``
- `if (!response.ok)`
- `useState` para `loading` y `error`

Ahora imaginen que el backend cambia y el token hay que mandarlo en otro header. ¿Cuántos archivos tendrían que tocar? ¿Y si se olvidan de uno?